

Dimostriamo che il problema del
Ciclo hamiltoniano $\bar{C}$ è in NP

INPUT: $G = (V, E)$

CERTIFICATO $\Rightarrow \langle v_{i1}, v_{i2}, \dots, v_{i|V|}, v_{i1} \rangle$
 $|V|$ vertici

l'algoritmo $A(x, y)$, input x, A
VERIFICATORE procede come

segue:

$\forall (v_{ij}, v_{ij+1}), 1 \leq j \leq |V|-1$
controlla che

- $(v_{ij}, v_{ij+1}) \in E$ (tempo costante)
- verifica che nessun vertice
di adiacenza del path

tra $v_{i1}, \dots, v_{i|V|}$

$\bar{C}$ è ripetuto 2 volte! $O(|E|)$

Problema CLIQUE $(G=(V,E), k)$

INPUT: $G=(V,E)$, k
(YES/NO) k (non orientato)
OUTPUT: esiste

un insieme $V' \subseteq V$, tale
che :

$$(1) |V'| = k$$

$$(2) \forall (u,v) \in V' \times V' - \{(v,v) : v \in V'\}$$

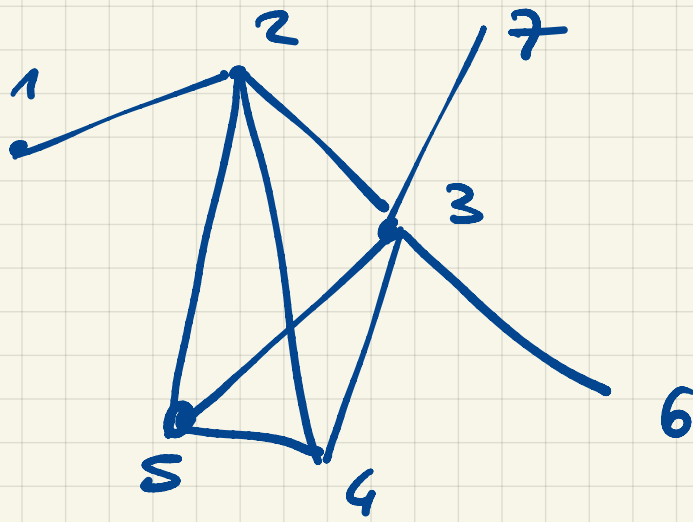
condizione $(u,v) \in E$

in alternative

$$(2) \forall (u,v) \text{ con } u,v \in V' \text{ (} v \neq u \text{)}$$
$$(u,v) \in E!$$

V' induce in G un
grafo completo!

esercizio



$K = 3$ esiste la clique di 3 vertici
 $K = 4$ $\{2, 3, 5\}$

esiste anche la clique di
 4 vertici $\{2, 3, 5, 4\}$

$K = 5$?

Dimostriamo che CLIQUE $\in$ NP!

Mostro che esiste un algoritmo

A verificatore che lavora

in tempo polinomiale nell'input

$\langle G = (V, E), k \rangle$

il certificato è un insieme
 di k vertici $\langle v_1, \dots, v_k \rangle$

che è una clique :

$\Rightarrow$ verifica è $O(\underline{K^2})$

perché controllo le coppie

$$(u, v) \in \{v_{i1}, \dots, v_{ik}\} \times \{v_1, \dots, v_{ik}\}$$

e verifico per le coppie

(u, v) con $u \neq v$ che

$(u, v) \in E$ (controlla $O(1)$)

il verificatore lavora in

tempo polinomiale in $O(K^2)$

SAT

satisfiability

soddisfizzabilità

- concetto di formula -

- connettivi logici

$\wedge$, $\vee$, $\neg$ (not $\rightarrow$ negazione)
AND OR

- VARIABILI BOOLEANA

variabile x_i che ha valori
in $\{0, 1\}$

- con le variabili booleane
creo i LETTERALI -

$l_i = x_i$ oppure

$$l_i = \neg x_i$$

il letterale $\bar{e}$: o la
variabile

• o $\neg$ variabile

I letterali l_i compaiono con
i connettivi $\vee, \wedge, -$

formano le **CLAUSOLE**

una clausola $\bar{e}$ la
combinazione in $\vee$ di
letterali

esempio • $C_1 = l_1 \vee l_2 \vee l_3$

• $C_2 = l_2 \vee l_3$
• $C_3 = l_3$

Φ formule $\bar{e}$ la
CONGIUNZIONE di clausole

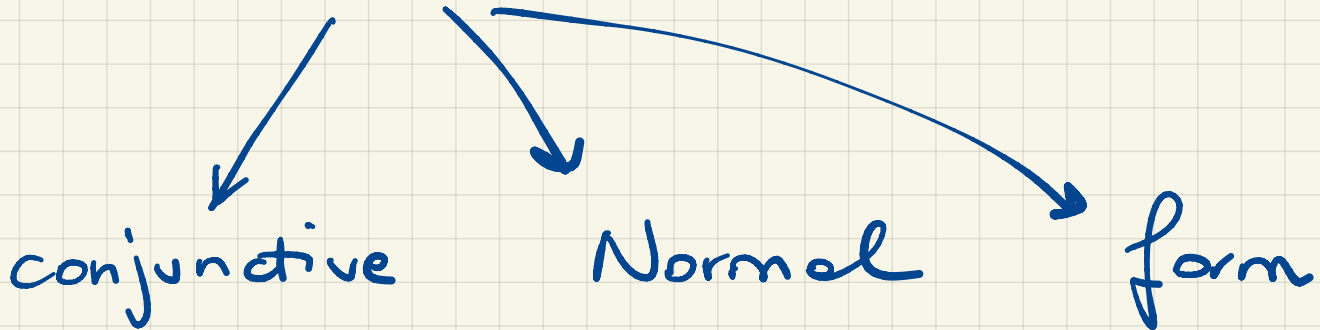
esempio

$$\phi_1 = c_1 \wedge c_2 \wedge c_3$$

$$\phi_2 = c_2 \wedge c_1$$

ϕ è detta anche

CNF -



K-CNF K è il numero di
letterali per clausola

2-CNF

$$c_i = l_j \vee l_k$$

3-CNF

$$c_i = l_1 \vee l_2 \vee l_3$$

K-SAT

Problema K-SAT (ϕ)

INPUT: ϕ che è una K-CNF

OUTPUT: 1 se ϕ è VERA
0 altrimenti

ϕ è VERA quando $\exists$ un
assegnamento di VERITÀ
per le variabili di ϕ

esempio

ϕ 2-SAT $c_1 \wedge c_2 \wedge c_3$

$$c_1 = x_1 \vee \neg x_1$$

$$c_2 = x_2 \vee x_1$$

$$c_3 = \neg x_2 \vee x_2$$

$$x_1 = 1, \quad x_2 = 1$$

c_1 è true, c_2 è true, c_3 è true

3-SAT è NP (è
tra i problemi difficili!)

2-SAT $\longrightarrow$ 3-SAT

dimostrare!

Dato un assegnamento di verità
basterà controllare che ogni
clausola è VERA (per
sostituzione dei valori nelle
variabili dei literal)

$\Downarrow$ polinomiale!

esempio

$$\phi = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2)$$

$c_1 \downarrow$
 $c_2 \uparrow$

CERTIFICATO • $x_1 = 0$, $x_3 = 1$
• $x_2 = 1$

- c_1 è vera per il letterale $x_3 = 1$
- c_2 è vera per il letterale $\neg x_1 = \neg 0 = 1$

Concetto

$\exists$ un problema di $\bar{E}$
capace di "RISOLVERE" tutti
i problemi in NP di $\bar{E}$

SAT (SAT $\bar{E}$ è una
"chiave di sotto-
procedura" di risoluzione
qualsiasi problema in NP)

ne consegue che se $\text{SAT} \in P$

allora $P = NP$ perché

SAT risolve in tempo

polinomiale tutti i problemi

in NP !

NB.

SAT non ha

un algoritmo polinomiale

se $P \neq NP$!

in alternativa

$\Rightarrow$ a meno che $P = NP$

SAT non ha un algoritmo polinomiale!